Generative AI Systems
Retrieval-Augmented Generation
RAG Variations

06
Retrieval-Augmented Generation

Introduction

In Chapter 5, we developed a chatbot capable of answering open-domain questions. However, these responses were limited to the knowledge stored in the model's parameters during training. This knowledge can become outdated, lacks domain specificity, and is prone to generating factually incorrect information (hallucinations).

Allowing chatbots to access this information improves the accuracy and relevance of their responses, especially for fact-based or specialized queries. A real-world example of such a system is Perplexity AI, an advanced search engine that uses retrieved sources to ground its answers.

Figure 1. Perplexity AI combines a search engine with a large language model to provide sourced answers.

Clarifying Requirements

Here's a typical interaction between a candidate and an interviewer:

Candidate: What are the current knowledge cutoffs for the models we're using?
Interviewer: The knowledge base includes company Wiki pages and a company-wide "Slack Overflow" vector store. The models' internal knowledge cutoffs are from mid-2023, so we need to incorporate recent data.

Candidate: Do the Wiki pages and Slack Overflow contain text, images, and other modalities?
Interviewer: For simplicity, let's focus on text. Adding support for other modalities, like images, for employing vision modalities do not need to be considered.

Candidate: Do the pages follow a fixed format or template?
Interviewer: No, the formats vary. Some are Wiki articles, others are single column, and others are mixed.

Candidate: Should the system support multi-turn conversations?
Interviewer: Yes, it should handle conversations with multiple turns.

Candidate: Is latency a concern for the system in real-time conversations?
Interviewer: Yes.

Candidate: Should the system respond in real time?
Interviewer: Users can tolerate a slight delay of a few seconds.

Candidate: Do we have ground truth relevance labels for evaluation?
Interviewer: Unfortunately, no. We'll need to rely on other metrics.

Candidate: Should the system support user feedback or follow-up questions?
Interviewer: Not initially. However, your design should be flexible to add support for feedback loops or follow-up questions.

Candidate: What is the expected growth in documents?
Interviewer: The document corpus is expected to grow by roughly 1000 pages per month.

Candidate: Do we need to ensure data privacy and security (e.g., restricting access based on employee status)?
Interviewer: Yes, we'll include user-level access controls in our discussion, but we'll keep the focus primarily on the core RAG pipeline.

Frame the Problem as an ML Task

Specifying the system's input and output

The problem statement for this chatbot closely resembles that of the open-domain QA task. The model processes this prompt alongside a continuously updated document database containing both text and images. The output is a text-based response that accurately addresses the user's query.

Figure 2. Chatbot prompt and response.

Pros

- **Customizable:** Prompting allows the model to generate responses tailored to specific domains.
- **Instruction-following:** Modern LLMs can follow detailed instructions, enhancing response accuracy and better aligns to business rules.

Cons

- **Computationally expensive:** Operating the entire knowledge base in prompt requires a lot of computational resources, which can be expensive.
- **Limited knowledge:** The approach implies frequent fine-tuning to continuously incorporate up-to-date data into the model.
- **Hallucinations:** Despite instructions, this approach struggles to ground responses in the provided data, making it susceptible to errors when it lacks the expertise to draw informed specialized knowledge.
- **Lack of attribution:** Without explicitly tracking sources, it becomes challenging to verify the accuracy of the model's responses.
- **Lack of references:** Fine-tuned models usually can't provide references for their answers, making it hard to verify or trace information sources.

Conclusion: Given the scale of our knowledge base and the need for accurate, verifiable responses, relying solely on prompting is not a viable approach. The computational cost alone makes it impractical, highlighting the need for a more scalable and efficient solution.

Prompt engineering

Advanced prompting guides, a general practice in LLM usage, can mitigate some of the limitations of this approach. Techniques such as chain-of-thought prompting, instruction following, and structured output specifications can significantly improve the model's reasoning and adherence to our specific requirements. However, these methods don't address the core issues of computational cost and knowledge currency. The model still processes the entire knowledge base with each query, and its knowledge is limited to what it was trained on or what can be crammed into the context window.

Figure 3. Prompt engineering approach.

Pros

- **Ease of use:** This prompt engineering is simple to use and requires no technical skills, which makes it suitable for a wide range of users.
- **Cost-effective for prototypes:** By leveraging a pretrained LLM, prompting avoids costly model training or fine-tuning, making it ideal for rapid prototyping and initial testing.
- **Flexibility:** The approach can easily incorporate additional data or instructions by simply adding more text to the prompt.

Cons

- **Inconsistency:** The quality and relevance of responses can vary greatly depending on how the prompt is phrased.
- **Limited customization:** The ability to tailor responses is restricted to the capabilities of the pretrained model and creativity of the prompt design.
- **Prompt engineering fatigue:** Crafting effective prompts can be time-consuming and requires a deep understanding of how the model interprets instructions.
- **Scalability issues:** Including all knowledge in prompts becomes impractical for large datasets, as it quickly exceeds the model's context window limits.

Conclusion: Prompting offers a quick and cost-effective approach to prototyping and leveraging pretrained LLM capabilities for knowledge-intensive tasks. However, its scalability limitations and inconsistent performance make it unsuitable for our production system. The approach's inability to handle our extensive knowledge base and the need for consistent, high-quality responses ultimately rules out this approach as a viable solution for our chatbot.

Choosing a suitable ML approach

Given the nature of the task, large language models (LLM) are well-suited for text generation and are often the default choice for knowledge-intensive tasks. We have already ruled out relying solely on the model's pretrained knowledge or providing all knowledge directly into context. The remaining LLM approach centers around two complementary strategies: fine-tuning and prompt engineering.

- Fine-tuning
- Prompt engineering
- Retrieval-augmented generation (RAG)

Let's explore each one in detail and discuss their trade-offs.

Fine-tuning

Fine-tuning involves adapting a pretrained general-purpose LLM to "internalize" our company's specific data, such as internal documents, by updating the model's parameters through additional training. This approach aims to enhance the model's performance on domain-specific tasks, such as answering questions about company policies or generating responses that adhere to internal guidelines and terminology.

Figure 4. Fine-tuning approach.

Pros

- **Customization:** Fine-tuning allows the model to generate responses tailored to specific domains.
- **Instruction following:** Fine-tuned models can better follow instructions, ensuring more accurate and better aligns to business rules.

Cons

- **Computationally expensive:** Updating the entire model's parameters requires a lot of computational resources, which can be expensive.
- **Limited knowledge:** This approach implies frequent fine-tuning to continuously incorporate up-to-date data into the model.
- **Hallucinations:** Despite instructions, this approach struggles to ground responses in the provided data, making it susceptible to errors when it lacks the expertise to draw informed specialized knowledge.
- **Lack of attribution:** Without explicitly tracking sources, it becomes challenging to verify the accuracy of the model's responses.
- **Lack of references:** Fine-tuned models usually can't provide references for their answers, making it hard to verify or trace information sources.

Conclusion: While fine-tuning offers customization and instruction following benefits, its computational cost, limited knowledge scalability, and inability to provide attributable responses make it unsuitable for our chatbot. The approach ensures the LLM generates responses that are both relevant and accurate based on the available information.

Retrieval-augmented generation (RAG)

RAG is an advanced method that combines the capabilities of a pretrained language LLM with a task-time retrieval system. Instead of relying solely on the model's internal knowledge, RAG enhances the LLM's input by dynamically retrieving relevant information from external documents, and feeding it into the model along with the original prompt. This approach ensures the LLM generates responses that are both relevant and accurate based on the available information.

In RAG systems, we refer to the prompt as the "augmented prompt" for clarity.

Figure 5. Retrieval-augmented generation (RAG).

Pros

- **Customization:** This approach allows the model to generate responses tailored to specific domains.
- **Instruction following:** Modern LLMs can follow detailed instructions, ensuring more accurate and better aligns to business rules.
- **Cost-effective:** By leveraging a pretrained LLM, prompting avoids costly model training or fine-tuning, making it ideal for rapid prototyping and initial testing.
- **Flexibility:** The system can easily incorporate additional data or instructions by simply adding more text to the prompt.
- **Attributable:** This approach allows tracking the model's responses back to the retrieved sources, enhancing transparency and verifiability.

Cons

- **Complexity:** This approach introduces additional components and processes to the system, increasing its overall complexity.
- **Latency:** The retrieval step can introduce latency, potentially affecting the system's real-time performance.

Conclusion: This approach effectively addresses the limitations of the other approaches we've discussed. It leverages the pretrained knowledge of the LLM while ensuring access to up-to-date, domain-specific information through retrieval. The ability to provide attributable responses and maintain control over the knowledge base makes it a robust and scalable solution for our chatbot.

Which approach is more suitable for ChatRDT?

The pretraining of the LLM provides the foundation for understanding and generating text. However, relying solely on this pretrained knowledge is insufficient for our chatbot, as it lacks access to our company's specific data and terminology. Among the remaining approaches, prompting offers a quick and cost-effective way to leverage the LLM's general capabilities for text generation. However, it struggles with scalability when dealing with our extensive knowledge base and may not consistently produce the specialized responses we need. Fine-tuning allows us to customize the model's behavior and improve its performance on our specific tasks, but it's computationally expensive and may still lead to hallucinations if not carefully managed. RAG emerges as a promising approach that addresses these limitations. It allows us to ground the LLM's responses in our company's data, ensuring access to relevant, up-to-date information while maintaining control over the knowledge base. This approach also enables attribution and traceability of the model's responses, which is crucial for building trust and accountability in our system.

Data Preparation

The performance of the RAG system relies on the quality of its knowledge database and the way it is indexed. When this database contains diverse data types like text, images, and various formats, preparing it for retrieving meaningful context is an important first step in building a reliable question-answering system, as discussed in Chapter 5.

In this section, we focus on preparing data from a collection of PDF pages. This involves a three-step process:

- Document parsing
- Document cleaning
- Document chunking

Document parsing

PDFs are one of the most widely used document formats. It is important to properly extract their content to prepare the data for indexing and ensuring that the LLM can correctly access and process information from these documents.

The two common strategies for parsing PDFs are:

- Text-only extraction
- Text + image extraction

However, even text-focused pipelines can benefit from incorporating vision capabilities, as many PDFs contain tables, charts, and other visual elements that pure text extraction methods might miss or misinterpret.

For this project, we'll explore both approaches to understand the trade-offs between simplicity and comprehensiveness.

Text-only extraction

Text-only extraction focuses on extracting machine-readable text from documents, ensuring the main textual content is accurately captured for indexing and retrieval. This approach works well for documents with a straightforward layout, such as those with a single column or clear reading order, where the primary goal is to preserve the document's overall structure. For example, LangChain has document-focused integrations, JTR (Jade Text Parser), and various other third-party integrations that support extracting text from various file formats, such as PDFs and HTML, while maintaining the hierarchical structure in Markdown.

Figure 6. Text-only extraction.

Pros

- **Simplicity:** Text-only extraction is straightforward to implement and maintain, making it a quick and reliable method for processing documents with simple layouts.
- **Speed:** This approach typically processes documents faster than methods involving vision capabilities, as it avoids the overhead of analyzing images and complex layouts.
- **Cost-effective:** Since it doesn't rely on vision models, text-only extraction is generally more cost-efficient, especially when dealing with large volumes of documents.

Cons

- **Limited to simple layouts:** This method works best with documents that have a straightforward reading order and may struggle with complex layouts, such as multi-column text or documents with embedded tables and charts.
- **Loss of visual information:** Important information conveyed through charts, graphs, or other visual elements will be lost or ignored.
- **Potential for errors:** The extraction process may misinterpret or skip over certain elements, especially if the document has unusual formatting or structure.

Text + image extraction

Text + image extraction leverages both text and vision models to extract and interpret both textual and visual information from documents. This approach is crucial when dealing with documents where important information is conveyed through charts, graphs, or other visual elements, or when the document's layout is complex or non-standard.

Figure 7. Text + image extraction.

Pros

- **Comprehensive:** This approach captures all the information in the document, including text, visual elements, and their relationships.
- **Handles complex layouts:** It can accurately process documents with multi-column layouts, embedded tables and charts, and other non-standard formatting.
- **Preserves context:** By understanding the relationship between text and visual elements, this method maintains the overall context and meaning of the document.

Cons

- **Complexity:** Implementing and maintaining a system that handles both text and image processing is more complex than text-only extraction.
- **Computationally expensive:** Processing images and running vision models can significantly increase the computational resources and time required.
- **Potential for errors:** The integration of text and image processing can introduce new sources of errors, especially if the system doesn't correctly associate related text and visual elements.

Document cleaning

Once we have identified the blocks of text, images, or tables in a document, the next step is to make them LLM-ready or machine-friendly. For text-based elements, this involves cleaning the extracted text to ensure it is in a format suitable for the LLM to process. This cleaning process may include:

- **Fixing broken Unicode:** This ensures proper rendering of special characters and prevents issues with text parsing.
- **Removing or replacing special characters:** Depending on the context, we may need to remove or replace certain special characters that can interfere with the LLM's understanding.
- **Normalizing whitespace:** Removing extra spaces, tabs, and line breaks can improve the model's ability to parse the text accurately.
- **Handling hyphenated words:** We may need to join words that were split across lines with hyphens.

Document cleaning addresses these challenges by breaking the text into smaller, manageable pieces or "chunks." Chunking helps ensure the quality and relevance of the retrieved context, as it allows us to retrieve only the most relevant information for the LLM to process.

Layout Parser offers a document-aware approach to chunking based on pretrained layout models. The following steps illustrate a typical workflow:

1. **Layout detection:** The parser uses pretrained detection models to identify various elements on a page, such as text blocks, paragraphs, and different types of visuals, including tables, charts, and figures.
2. **Element classification:** The parser classifies the detected elements into specific categories, such as paragraphs, titles, lists, tables, and various types of visuals (e.g., bar charts, line charts, pie charts).
3. **Structured output generation:** The parser produces a structured output containing key types of data:
   - **Text blocks:** Includes the coordinates of bounding boxes, the extracted text, and the hierarchical relationships between elements.
   - **Non-text elements:** Includes the coordinates of figures or images.

Figure 8. Layout parser workflow.

Several online services provide document parsing services, for example, Google Cloud Document AI, OCR and PDF-to-JSON. These services offer pre-trained models and cloud infrastructure to handle large-scale document processing, ensuring high accuracy and efficient text extraction.

Document chunking

Once we have identified the blocks of text, images, or tables in a document, the next step is to make them LLM-ready or machine-friendly. For text-based elements, this involves cleaning the extracted text to ensure it is in a format suitable for the LLM to process. This cleaning process may include:

- **Fixing broken Unicode:** This ensures proper rendering of special characters and prevents issues with text parsing.
- **Removing or replacing special characters:** Depending on the context, we may need to remove or replace certain special characters that can interfere with the LLM's understanding.
- **Normalizing whitespace:** Removing extra spaces, tabs, and line breaks can improve the model's ability to parse the text accurately.
- **Handling hyphenated words:** We may need to join words that were split across lines with hyphens.

Document cleaning addresses these challenges by breaking the text into smaller, manageable pieces or "chunks." Chunking helps ensure the quality and relevance of the retrieved context, as it allows us to retrieve only the most relevant information for the LLM to process.

Layout Parser offers a document-aware approach to chunking based on pretrained layout models. The following steps illustrate a typical workflow:

1. **Layout detection:** The parser uses pretrained detection models to identify various elements on a page, such as text blocks, paragraphs, and different types of visuals, including tables, charts, and figures.
2. **Element classification:** The parser classifies the detected elements into specific categories, such as paragraphs, titles, lists, tables, and various types of visuals (e.g., bar charts, line charts, pie charts).
3. **Structured output generation:** The parser produces a structured output containing key types of data:
   - **Text blocks:** Includes the coordinates of bounding boxes, the extracted text, and the hierarchical relationships between elements.
   - **Non-text elements:** Includes the coordinates of figures or images.

Figure 8. Layout parser workflow.

Several online services provide document parsing services, for example, Google Cloud Document AI, OCR and PDF-to-JSON. These services offer pre-trained models and cloud infrastructure to handle large-scale document processing, ensuring high accuracy and efficient text extraction.

Document chunking

Once we have identified the blocks of text, images, or tables in a document, the next step is to make them LLM-ready or machine-friendly. For text-based elements, this involves cleaning the extracted text to ensure it is in a format suitable for the LLM to process. This cleaning process may include:

- **Fixing broken Unicode:** This ensures proper rendering of special characters and prevents issues with text parsing.
- **Removing or replacing special characters:** Depending on the context, we may need to remove or replace certain special characters that can interfere with the LLM's understanding.
- **Normalizing whitespace:** Removing extra spaces, tabs, and line breaks can improve the model's ability to parse the text accurately.
- **Handling hyphenated words:** We may need to join words that were split across lines with hyphens.

Document cleaning addresses these challenges by breaking the text into smaller, manageable pieces or "chunks." Chunking helps ensure the quality and relevance of the retrieved context, as it allows us to retrieve only the most relevant information for the LLM to process.

Layout Parser offers a document-aware approach to chunking based on pretrained layout models. The following steps illustrate a typical workflow:

1. **Layout detection:** The parser uses pretrained detection models to identify various elements on a page, such as text blocks, paragraphs, and different types of visuals, including tables, charts, and figures.
2. **Element classification:** The parser classifies the detected elements into specific categories, such as paragraphs, titles, lists, tables, and various types of visuals (e.g., bar charts, line charts, pie charts).
3. **Structured output generation:** The parser produces a structured output containing key types of data:
   - **Text blocks:** Includes the coordinates of bounding boxes, the extracted text, and the hierarchical relationships between elements.
   - **Non-text elements:** Includes the coordinates of figures or images.

Figure 8. Layout parser workflow.

Several online services provide document parsing services, for example, Google Cloud Document AI, OCR and PDF-to-JSON. These services offer pre-trained models and cloud infrastructure to handle large-scale document processing, ensuring high accuracy and efficient text extraction.

Indexing

After preparing the data through parsing and chunking, the final critical step in the RAG system is indexing. Indexing is the process of organizing the processed data into a structure that allows for efficient retrieval of relevant information. This step is crucial to ensuring that the system can quickly access the most relevant chunks of information when a query is made.

To determine the indexing process, it is crucial to understand various retrieval techniques and choose the one that works best. Popular sparse retrieval techniques include:

- Keyword based
- TF-IDF
- Knowledge graph based
- Vector based

Let's first explore each technique, and then index our data to enable efficient retrieval.

Keyword based

Traditional keyword based retrieval relies on matching exact query terms with the content of documents. It's fast and straightforward, cannot understand the meaning of queries. For example, it may struggle with synonyms, leading to incomplete or irrelevant results. This method's effectiveness depends on how well the query phrasing matches the exact words in the document, making it sensitive to variations in language and terminology.

TF-IDF

TF-IDF (Term Frequency-Inverse Document Frequency) is an enhancement over keyword based retrieval. It assigns weights to words based on their frequency in a document and across the entire corpus. This method helps to identify more significant terms in each document, making them more useful for dealing with complex documents.

Knowledge graph based

Knowledge graph based retrieval stores information as a network of interconnected entities and relationships. This approach allows for understanding complex relationships between different pieces of information. It's particularly useful for queries that require understanding connections between various entities or concepts. However, building and maintaining a knowledge graph can be resource-intensive, and its effectiveness depends on the quality and completeness of the underlying knowledge base.

Vector based

Instead of relying on keyword matching, vector based retrieval uses dense vector embeddings to represent both queries and documents in a high-dimensional space. This method can understand the semantic meaning of queries and find relevant information even when the exact words don't match. It's quite flexible and can handle various types of data, including text and images, making it a versatile approach for large-scale retrieval systems.

Which retrieval technique is suitable for the ChatRDT?

Each of these approaches has its strengths and weaknesses. For our system, we'll explore the methods of dense vector based retrieval. In this case, the embeddings (vectors) are generated by a neural network trained to capture the semantic meaning of data. This allows for more nuanced and context-aware retrieval, understanding the deeper meanings and relationships between different pieces of information. This approach is powerful for large-scale retrieval systems, as it can handle various types of data and maintain high accuracy in finding relevant information.

Indexing data for vector-based retrieval

In a vector-based retrieval system, each chunk of data is converted into an embedding vector representing the content in a numerical format. When indexing, these vectors are assigned to compute the embeddings and store them in a vector database. This allows for efficient retrieval based on semantic similarity. The system can quickly find the most relevant information by comparing the query vector with the stored vectors. This approach enables more sophisticated and context-aware information retrieval, which is crucial for applications like question-answering systems or recommendation systems where understanding the meaning and context of the data is more important than exact keyword matches.

Figure 9. Indexing process for text and image chunks using embedding models and a vector database.

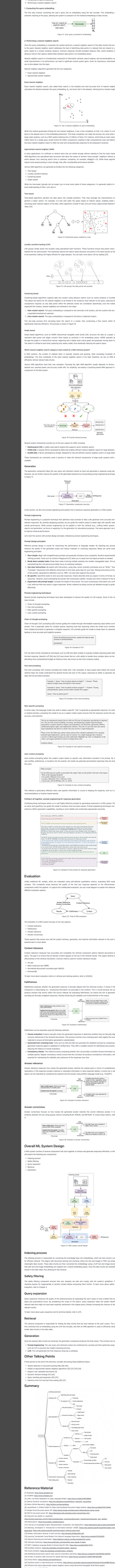
In summary, we use a three-step process to prepare PDFs for the RAG system. First, we apply document parsing techniques to convert the PDF into a machine-friendly format while preserving its layout and structure. Next, we clean the extracted chunking to ensure high-quality text, ensuring, inappropriate chunks. Finally, each chunk is converted to an embedding vector and indexed individually into a vector database, enabling efficient retrieval based on semantic similarity.

Model Development

Architecture

This section explores the architecture of a RAG system, focusing on the NLI, modules used in the indexing, retrieval, and generation components.

Text-to-text
Text-to-image
Indexing
Retrieval
Generation
Text-to-text
Text-to-image
Text-to-text
Text-to-image
Text-to-text
Text-to-image


Chat prompt
Text encoder
Chat prompt embedding
Text encoder
Chat prompt embedding
Text encoder
Text encoder
Text encoder

[38] TableNet: Deep Learning model for end-to-end Table detection and Tabular data extraction from Scanned Document Images. https://arxiv.org/abs/2001.01469.

[39] CascadeTabNet: An approach for end to end table detection and structure recognition from image-based documents. https://arxiv.org/abs/2004.12629.

[40] Deepdesrt: Deep learning for detection and structure recognition of tables in document images. https://ieeexplore.ieee.org/document/8270123.

[41] Active Retrieval Augmented Generation. https://arxiv.org/abs/2305.06983.

[42] Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. https://arxiv.org/abs/2310.11511.

[43] Precise Zero-Shot Dense Retrieval without Relevance Labels. https://arxiv.org/abs/2212.10496.

Footnotes

1. Accurate at the time of writing. ↩
2. Specifics are unknown to the public at the time of writing. ↩

Mark as Complete

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.

Ask Alex